{EPITECH}

MY_TOP_

< LET'S TAKE A LOOK AT UNIX PROCESSES ! />

MY_TOP

binary name: my_top
language: C
compilation: via Makefile, including re, clean and fclean rules
Forbidden functions: system, exec*, popen, getloadavg, getrusage, getrlimit, getutent, setutent or any other function that retrieves process or system information for you.

- ✓ The totality of your source files, except all useless files (binary, temp files, objfiles,. ..), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.
- ✓ Error messages have to be written on the error output, and the program should then exit with the 84 error code (0 if there is no error).

{EPITECH}

The project

Objectives

You must recode the **top** command.

It is a tool to monitor your system and processes.

Your project will need to use the Terminal User Interface library **ncurses**.

Here is an example of what the `top` command output looks like:

```
top - 10:19:23 up  3:06,  0 user,  load average: 2.01, 1.49, 1.19
Tasks:   6 total,   2 running,   4 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.4 us,  7.3 sy,  2.7 ni, 89.5 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  11950.5 total,   8126.7 free,   2742.2 used,   1316.7 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   9208.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   20 clery     23   3  372764   4932   2412 R  99.7   0.0   1:42.90 yes
    1 root      20   0  375128   7608   4168 S   0.0   0.1   0:00.07 bash
   16 root      20   0  376336   7548   4388 S   0.0   0.1   0:00.05 su
   17 clery     20   0  369924   4888   2360 S   0.0   0.0   0:00.02 sh
   18 clery     20   0  375692   9988   3976 S   0.0   0.1   0:00.11 bash
   29 clery     20   0  379284   9084   3956 R   0.0   0.1   0:00.03 top
```

Parameters

You **must** handle the following options, **in any order**:

* `-U <username>`: allows to filter the processes shown by username.
* `-d <seconds[.cents]>`: modifies the delay between refreshes (in seconds. Default: 3.0).
* `-n <frames>`: defines how many frames are to be shown before the program exits (default: unlimited).

These options should behave in the same way that they do with **top**

The following example shows 10 frames of processes owned by user `clery` with 1.5 second interval between frames.

Terminal

```
$> ./my_top -U clery -n 10 -d 1.5
```

The following example just shows processes owned by user `clery`, until the program is manually stopped, with 3 second interval.

Terminal

```
$> ./my_top -U clery
```

```
top - 10:39:58 up  3:27,  0 user,  load average: 0.95, 1.01, 1.17
Tasks:   5 total,   1 running,   4 sleeping,   0 stopped,   0 zombie
%Cpu(s): 11.8 us,  5.9 sy,  0.0 ni, 82.4 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  11950.5 total,   8180.0 free,   2685.3 used,   1320.4 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   9265.2 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   26 clery     20   0  369924   4884   2360 S   0.0   0.0   0:00.02 sh
   27 clery     20   0  375116   7544   4104 S   0.0   0.1   0:00.07 bash
   29 clery     20   0  379272   8952   3952 R   0.0   0.1   0:00.04 top
```

Features

Your program **must** be able to retrieve system information, and process statistics.

In the upper section of your ncurses window, you **must** display the system informations.
Below that, you **must** display an array of individual process statistics.

In the upper section you **should** display as much system informations possible, including:

- ✓ Time of day
- ✓ How long the laptop has been up (powered on)
- ✓ How many users are currently logged in
- ✓ Load average
- ✓ The amount of Tasks total, running, sleeping, stopped or zombie
- ✓ CPU usage
- ✓ Memory usage
- ✓ Swap usage

In the lower section, you **should** display as much process informations possible, including:

- ✓ **PID** of the process
- ✓ **USER** owning that process
- ✓ **PR**iority
- ✓ **NI**ce Value
- ✓ **VIRT**ual Memory Size
- ✓ **RES**ident Memory Size
- ✓ **SH**a**R**ed Memory Size
- ✓ Process **S**tatus
- ✓ **CPU** percent usage
- ✓ **MEM**ory percent usage
- ✓ **TIME** since the process has started
- ✓ The **COMMAND** name

{EPITECH}

```
top - 09:04:31 up 3 days, 21:03,  0 users,  load average: 1.29, 1.00, 0.95
Tasks:   2 total,   1 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.5 us,  0.2 sy,  0.0 ni, 99.3 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  11952.6 total,    608.4 free,   1884.6 used,   9459.6 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   9821.5 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   25 root      20   0  376216   6824   3456 R   0.7   0.1   0:00.04 top
    1 root      20   0  375108  10140   4352 S   0.0   0.1   0:00.05 bash
```

The image above estimates how difficult the different sections are, although some of you may find your personal experience to be different.

This can be due to multiple factors : the research needed to find out how to calculate the results, how to handle data to make it change over time, or any other reason that would make something difficult.

The uptime value can be tricky, here is how it works:

- ✓ it prints days only if up for at least 24 hours.
- ✓ it prints hours only if there is hours is more than 0.
- ✓ if no hours are printed, it print `30 min`, otherwise, `2:30`.

  So, if the system has been up for 48 hours and 30 min, you would print `up 2 days, 30 min`.

You should probably start with the easiest columns. CPU and MEM usage are not that easy !

Formatting exactly like `top` is not mandatory.

The following keyboard interactions are also expected:

- ✓ Typing **E should** cycle through different units of memory for processes
  Shift+E should cycle through different units of memory for the system section
  - KiB
  - MiB
  - GiB
  - TiB
  - PiB
  - EiB (only for the system section)
- ✓ Using **up and down arrows must** allow you to scroll through your process list
- ✓ Typing **K must** open a prompt to send a signal to a process
  - By default, it **must** offer to send the signal to the **highest process in your list**
  - By default, it **must** offer to send the `SIGTERM` signal
  - You are not required to provide line edition

man top
man procfs
top -> hit H on your keyboard to open the documentation

Scrolling down or up changes the **highest**, or **selected** process in your list. So the default PID to send a signal to with the K command **should change** !

Sending signals is an important feature. Do not forget about it.

But, really, I mean it. Read the manuals for **top** and **procfs**. I swear they're useful.

Warning

For the moulinette to work, you **must** respect a few things:

You **MUST** implement the PID column before any other process column, without it, your project will not be evaluated.

For processes, the "title bar" **must** match the values underneath. So for each name in the title bar, a value **must** be found in the process infos. Eg:

All data is aligned with its header name, so it's ok

```
PID USER PR

1 root 20
27 root 20
33 root 20
34 root 20
123 root 20
218 root 20
231 root 20
241 root 20
321 root 20
343 root 20
357 root 20
408 root 20
10127 root 20
10880 root 20
21875 root 20
67201 root 20
67225 root 20
69249 root 20
69963 root 20
70348 root 20
72524 root 20
72574 root 20
72642 root 20
```

PR is the 3rd header name, but 2nd value, so it's not ok

```
PID USER PR

1       20
27       20
33       20
34       20
123       20
218       20
231       20
241       20
321       20
343       20
357       20
408       20
10127       20
10880       20
21875       20
67201       20
67225       20
69249       20
69963       20
70348       20
72524       20
72574       20
72586       20
```

On the system statistics side, make sure that every word that is not a value is the same as in `top`.

Once again, remember: software development is an exact science. It's important to avoid typos, and other kinds of issues that might cause trouble with the moulinette's tests.

Bonus

There's so many possible bonuses it would be longer to enumerate them than writing the rest of the subject... But here are a few ideas:

- ✓ Sort rows by column value
- ✓ Using **<** and **>** could change which column the array is sorted by (default PID)
- ✓ Typing **Shift+R** could change the sorting direction
  - You could sort from highest to lowest value by default
- ✓ More columns ? **PPID, GROUP, UID, GID**. There's so many, just read the man you'll find some easy ones
- ✓ Scrolling horizontally ? Try it out in `top`, it's probably not that hard.
- ✓ Did you try the **Z** command ? Some colors would make that window look better.
- ✓ Forest mode ? **Shift+V**
- ✓ Threads ? **Shift+H**
- ✓ Show the entire command line ? **C**
- ✓ ...
- ✓ **htop** ?...

{EPITECH}

Unit tests

Testing a project which is using external libraries and system calls can be tedious, but it is possible to organize a project in such a way that external calls are isolated, and the core functionalities are easier to test. Also, architecturing your project to make it easily testable from the beginning usually leads to cleaner code.

Here is an example. Let's say we have a `my_top.h` file:

```
#ifndef MY_TOP_H_
    #define MY_TOP_H_

extern const char *LOADAVG_PATH;

typedef struct system_s {
    float loadavg[3];
    // Some fields
} system_t;

#endif
```

This header defines a `LOADAVG_PATH` variable to be used within our my_top to use this path as the `loadavg` file to read.

We can override this variable within our tests to specify a fixture file.

```
#include <criterion/criterion.h>
#include "my_top.h"

Test(my_top, check_loadavg)
{
    int err;
    system_t *system;

    LOADAVG_PATH = "./fixtures/loadavg";
    system = fetch_system_data(); // This function uses LOADAVG_PATH internally
    cr_assert_neq(system, NULL, "An error occured, fetch_system_data returned NULL");
    cr_assert_eq(system->loadavg[0], 2);
    cr_assert_eq(system->loadavg[1], 3);
    cr_assert_eq(system->loadavg[2], 4);
}
```

This way, we can test that the data parsing part of our my_top works the way it should, and we can be alerted by our unit tests if any other change in our code would break this functionality.

We could test other parts, such as formatting of said data, or refreshing this same data, and ensure that those work as well, without ever testing ncurses code.

v 1.0.0